

Chapter 3 Summary — CFR Variants & Monte Carlo Methods

Research on the possibilities for applying Artificial Intelligence in computer games

Alexander Andreev

April 2026

Contents

3 Chapter 3 — CFR Variants & Monte Carlo Methods	1
3.1 Why Vanilla CFR Breaks at Scale	1
3.2 Monte Carlo Tree Search as Inspiration	2
3.3 Markov Chains and the Law of Large Numbers	3
3.4 Leduc Poker — Scaling Up the Benchmark	4
3.5 CFR+ — The Regret Flooring Trick	5
3.6 MCCFR — Sampling the Game Tree	7
3.7 The Variance-Speed Tradeoff	10
3.8 When MCCFR Wins — The Crossover Point	13
3.9 Connections to Chapter 2 and Forward Pointers	15

3 Chapter 3 — CFR Variants & Monte Carlo Methods

3.1 Why Vanilla CFR Breaks at Scale

Chapter 2 established a working Vanilla CFR solver on Kuhn Poker — 12 information sets, 58 nodes (30 terminal), solvable in milliseconds. That experience is deceptive. Vanilla CFR requires a **full traversal of the game tree on every iteration**, and real games are not Kuhn. Leduc Poker has 936 information sets (~78× larger); Limit Texas Hold'em has over 10^{14} , and a single traversal of its tree took about an hour on 4,800 CPU cores (see Section 3.8).

Chapter 3 closes the scale gap through two complementary mechanisms, developed independently in the game-theory literature and later combined in practice:

1. **CFR+ (Tammelin, 2014)**¹ — a modification of vanilla CFR that achieves dramatically faster convergence through regret flooring, linear strategy averaging, and alternating updates. CFR+ was the algorithm that “solved” Heads-Up Limit Texas Hold'em in 2015 — the first non-trivial poker variant to be essentially solved.

¹Tammelin, O. (2014). “Solving Large Imperfect Information Games Using CFR+.” arXiv:1407.5042. The Heads-Up Limit Hold'em result is Bowling, M., Burch, N., Johanson, M. & Tammelin, O. (2015). “Heads-up limit hold'em poker is solved.” *Science*, 347(6218), 145-149.

2. **Monte Carlo CFR (Lanctot et al., 2009)**² — instead of traversing the entire game tree each iteration, MCCFR samples a small portion. Each iteration is orders of magnitude cheaper, at the cost of noisier updates. It is the main approach for games where full traversal is physically impossible; the other, game abstraction, is the subject of Chapter 4.

For the thesis, these solvers compute the baseline Nash strategy — the “play it safe” anchor from which an adaptive agent deviates based on opponent observations (Contribution #1). On games of Kuhn and Leduc size that is CFR+ (see Section 3.7); MCCFR becomes necessary on larger games. CFR+ makes that computation tractable on the intermediate benchmark games used in Chapters 5–8.³

3.2 Monte Carlo Tree Search as Inspiration

Monte Carlo Tree Search (MCTS) is the canonical family of algorithms that use **random sampling** to evaluate positions in a game tree without exhaustive enumeration. Its design philosophy — “evaluate some branches to full depth instead of all branches to fixed depth” — directly motivates MCCFR.

MCTS builds an asymmetric search tree through four repeated phases:

1. **Selection** — descend from the root using a policy (typically UCB1, Upper Confidence Bound) that balances exploitation of known-good moves with exploration of under-visited ones.
2. **Expansion** — at a leaf, add one or more child nodes to the tree.
3. **Simulation (rollout)** — from the new node, play a random game to completion.
4. **Backpropagation** — propagate the terminal result back up the visited path, updating visit counts and cumulative rewards.

After many iterations, the child of the root with the highest visit count is selected as the best move. Guided by the selection policy, the playouts, aggregated over thousands of iterations, converge to accurate value estimates — even without any domain knowledge beyond the game rules.

Classical alternatives like **Minimax with alpha-beta pruning** evaluate every relevant node to a fixed depth, then apply a heuristic at the leaves. Both approaches have their regimes:

Table 1: Minimax with alpha-beta pruning against MCTS, across five properties.

Property	Minimax + α - β	MCTS
Tree coverage	All branches to depth d	Selected branches to terminal

²Lanctot, M., Waugh, K., Zinkevich, M. & Bowling, M. (2009). “Monte Carlo Sampling for Regret Minimization in Extensive Games.” *Advances in Neural Information Processing Systems 22*, 1078-1086.

³Bowling, M., Burch, N., Johanson, M. & Tammelin, O. (2015). “Heads-up limit hold’em poker is solved.” *Science*, 347(6218), 145–149. DOI 10.1126/science.1259433. Used CFR+ to solve heads-up limit Texas Hold’em — the first non-trivial imperfect-information game played competitively by humans to be essentially solved.

Property	Minimax + α - β	MCTS
Evaluation	Heuristic at depth limit	Exact (terminal payoff) or neural net
Branching factor sensitivity	Exponential: $O(b^d)$	Graceful: focuses on promising branches
Domain knowledge needed	Strong eval function	None (random rollout) or learned
Best suited for	Moderate branching, good heuristics	High branching, weak/no heuristics

For Go (branching factor ~ 250 , no good evaluation function before neural networks), Minimax is impractical. MCTS was the breakthrough that made Go AI competitive; the same “sample instead of enumerate” principle carries over to imperfect-information game trees via MCCFR.⁴

3.3 Markov Chains and the Law of Large Numbers

The “Monte Carlo” in MCCFR refers to a broad family of computational methods that use **repeated random sampling** to obtain numerical results. The method was devised at Los Alamos in 1946, where Stanislaw Ulam and John von Neumann used random sampling to simulate neutron diffusion — a problem too complex for analytical solution; the name, suggested by Nicholas Metropolis, refers to the Monte Carlo casino.⁵

The mathematical foundation rests on **Markov chains** — stochastic processes where the next state depends only on the current state, not on the history of how it was reached (the Markov property). A game tree traversal is a Markov chain: from any game state, the transition depends only on the current state and the action chosen.

The key theoretical guarantee is the **Law of Large Numbers**: if enough trajectories are drawn independently, the average of the sampled values converges to the true expected value. For MCCFR, at a fixed strategy profile:

$$\frac{1}{T} \sum_{t=1}^T \hat{v}_I^{(t)} \xrightarrow{T \rightarrow \infty} v_I$$

⁴Browne, C. et al. (2012). “A Survey of Monte Carlo Tree Search Methods.” *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 1–43.

⁵Metropolis, N. (1987). “The Beginning of the Monte Carlo Method.” *Los Alamos Science*, 15, 125ff. The first publication is Metropolis, N. & Ulam, S. (1949). “The Monte Carlo Method.” *JASA*, 44(247), 335–341.

where $\hat{v}_I^{(t)}$ is the sampled counterfactual value at information set I on iteration t , and v_I is the true value that vanilla CFR computes exactly. The sampled values are **unbiased estimators** — their expected value equals the true value — and, with sampling probabilities bounded away from zero, this gives with probability at least $1 - p$ the same $O(1/\sqrt{T})$ decrease of average regret as full-traversal CFR, hence convergence of the average strategy to an approximate Nash equilibrium.²

The cost is **variance**. Individual samples can deviate substantially from the true value, requiring more iterations to reach the same precision. That variance-for-speed tradeoff is the central theme of this chapter.⁶

3.4 Leduc Poker — Scaling Up the Benchmark

Kuhn Poker captures the essence of imperfect information in the smallest possible game tree. Leduc Poker scales that up by roughly two orders of magnitude — still solvable exactly, but large enough that algorithm differences become meaningful:

Table 2: Kuhn and Leduc Poker compared: cards, betting rounds, information sets and tree size.

Property	Kuhn Poker	Leduc Poker
Cards	3 (J, Q, K)	6 ($\{J, Q, K\} \times \{\spadesuit, \heartsuit\}$)
Rounds	1	2 (pre-flop + community card)
Community card	None	1 revealed between rounds
Hand ranking	High card only	Pair (private = community) beats high card
Bet sizes	Fixed (1 chip)	Variable (2 in round 1, 4 in round 2)
Max raises/round	1	2
Information sets	12	936
Game tree nodes	58	10,200
Chance outcomes	6	120

Three qualitatively new features appear:

- **Multi-round structure** — information changes between rounds (community card reveal), requiring strategies that adapt to new information.
- **Pair hands** — the hand ranking now depends on the community card; a Jack can become the best hand if a Jack is dealt on the board.

⁶Sutton, R.S. & Barto, A.G. (2018). *Reinforcement Learning: An Introduction*, 2nd edition. MIT Press. Ch. 1 (the field); Ch. 3 (finite Markov decision processes); Ch. 4 (dynamic programming); Ch. 5 (Monte Carlo methods); Ch. 6 (temporal-difference learning). <http://incompleteideas.net/book/the-book-2nd.html>

- **Larger bet sizes in later rounds** — the 4-chip raise in round 2 means decisions in later rounds carry more weight.

Despite being $\sim 78\times$ larger than Kuhn, Leduc remains small enough for exact computation (a full tree traversal takes ~ 50 ms). That makes it the ideal benchmark: large enough for performance differences to be meaningful, small enough that all algorithms can be run to near-convergence within minutes.⁷

3.5 CFR+ — The Regret Flooring Trick

CFR+ makes three small changes to vanilla CFR, each trivial to implement but dramatic in effect.

Regret flooring. In vanilla CFR, cumulative regrets can become arbitrarily negative:

$$R^{T+1}(I, a) = R^T(I, a) + r^T(I, a)$$

CFR+ floors regrets at zero after every update:

$$R^{T+1}(I, a) = \max(R^T(I, a) + r^T(I, a), 0)$$

This prevents actions from accumulating large negative regret that takes many iterations to “pay off” before the action is reconsidered. The analogy to ReLU activations in neural networks is direct: both clip negative values to zero.

Linear strategy averaging. Vanilla CFR weights every iteration’s strategy equally in the running average. CFR+ weights iteration t by t itself:

$$\bar{\sigma}^T(I, a) = \frac{\sum_{t=1}^T t \cdot \sigma^t(I, a)}{\sum_{t=1}^T t}$$

Later iterations — which have lower regret and better strategies — contribute more to the average. Early, noisy iterations are down-weighted.

Alternating updates. Instead of updating both players simultaneously from the same strategy profile, CFR+ updates them in turn within each iteration, so the second player’s update already sees the first player’s new strategy. The work per iteration is unchanged; the benefit is faster convergence.

Combined effect: empirical convergence improves from $O(1/\sqrt{T})$ to approximately $O(1/T)$. On Leduc, CFR+ reaches 2.0×10^{-4} after about 1,100 iterations; at vanilla CFR’s measured rate the same value would take on the order of two million iterations (extrapolated). This is what made it feasible to solve

⁷Southey, F. et al. (2005). “Bayes’ Bluff: Opponent Modelling in Poker.” *UAI*. arXiv:1207.1411.

Heads-Up Limit Texas Hold'em in 2015. Note: the $O(1/T)$ rate is consistently observed but unproven; what is proven is the family-wide $O(1/\sqrt{T})$ bound,⁸ whose original proof was corrected by Burch et al.⁹

Exploitability here is the average best-response gain against the two players, $(BR_0 + BR_1)/2$; NashConv is their sum, i.e. twice as large. Empirically, on Leduc after 5,000 iterations OpenSpiel's CFR+ reaches exploitability $\approx 1.8 \times 10^{-5}$ while vanilla CFR is still at 3.6×10^{-3} — a $\sim 190\times$ improvement from two of the three modifications (both OpenSpiel solvers already alternate updates).¹

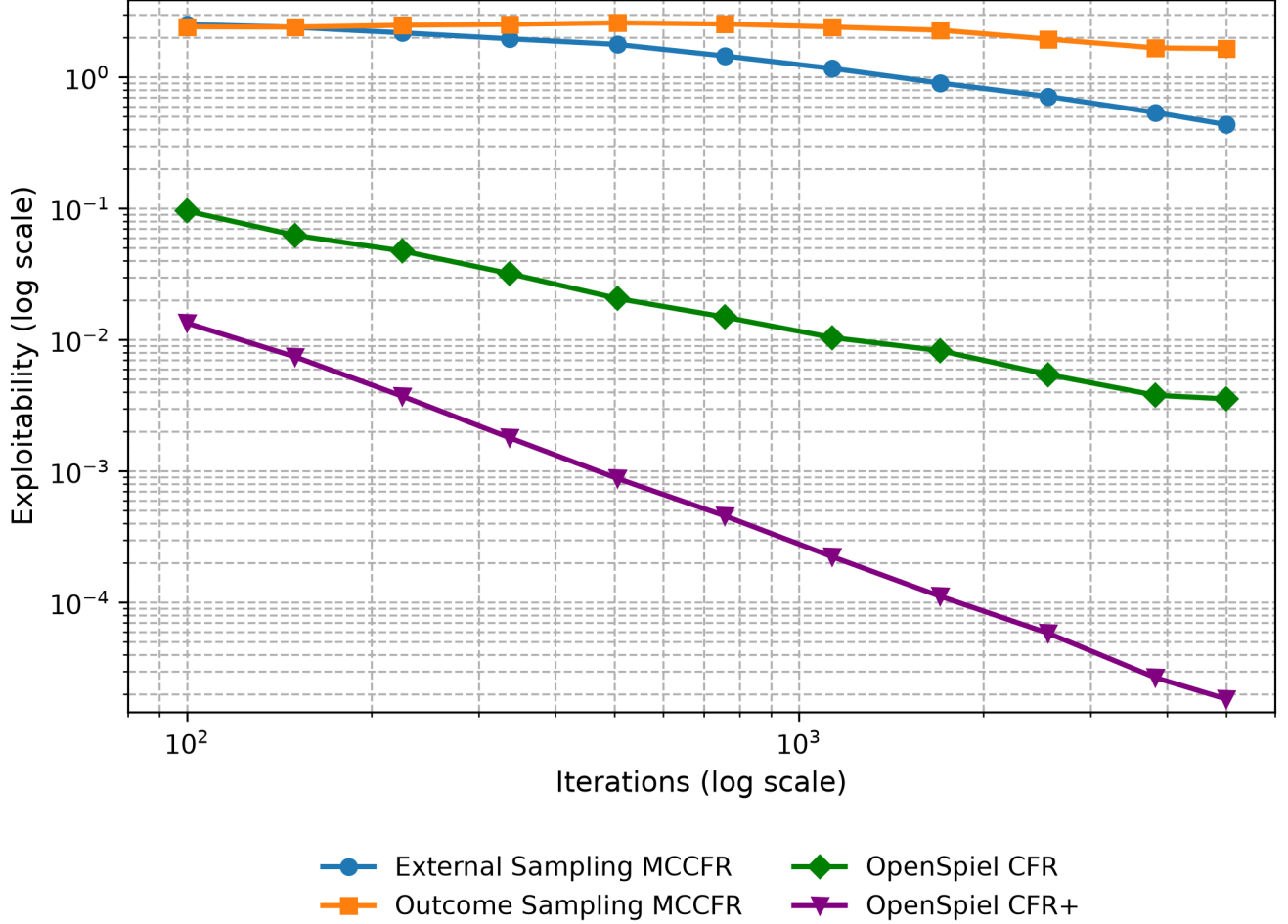


Figure 1: Leduc Poker: exploitability vs iterations for the four OpenSpiel solvers

⁸Tammelin, O., Burch, N., Johanson, M. & Bowling, M. (2015). “Solving Heads-Up Limit Texas Hold'em.” *Proc. IJCAI*, 645–652.

⁹Burch, N., Moravčík, M. & Schmid, M. (2019). “Revisiting CFR+ and Alternating Updates.” *Journal of Artificial Intelligence Research*, 64, 429–443. DOI 10.1613/jair.1.11370.

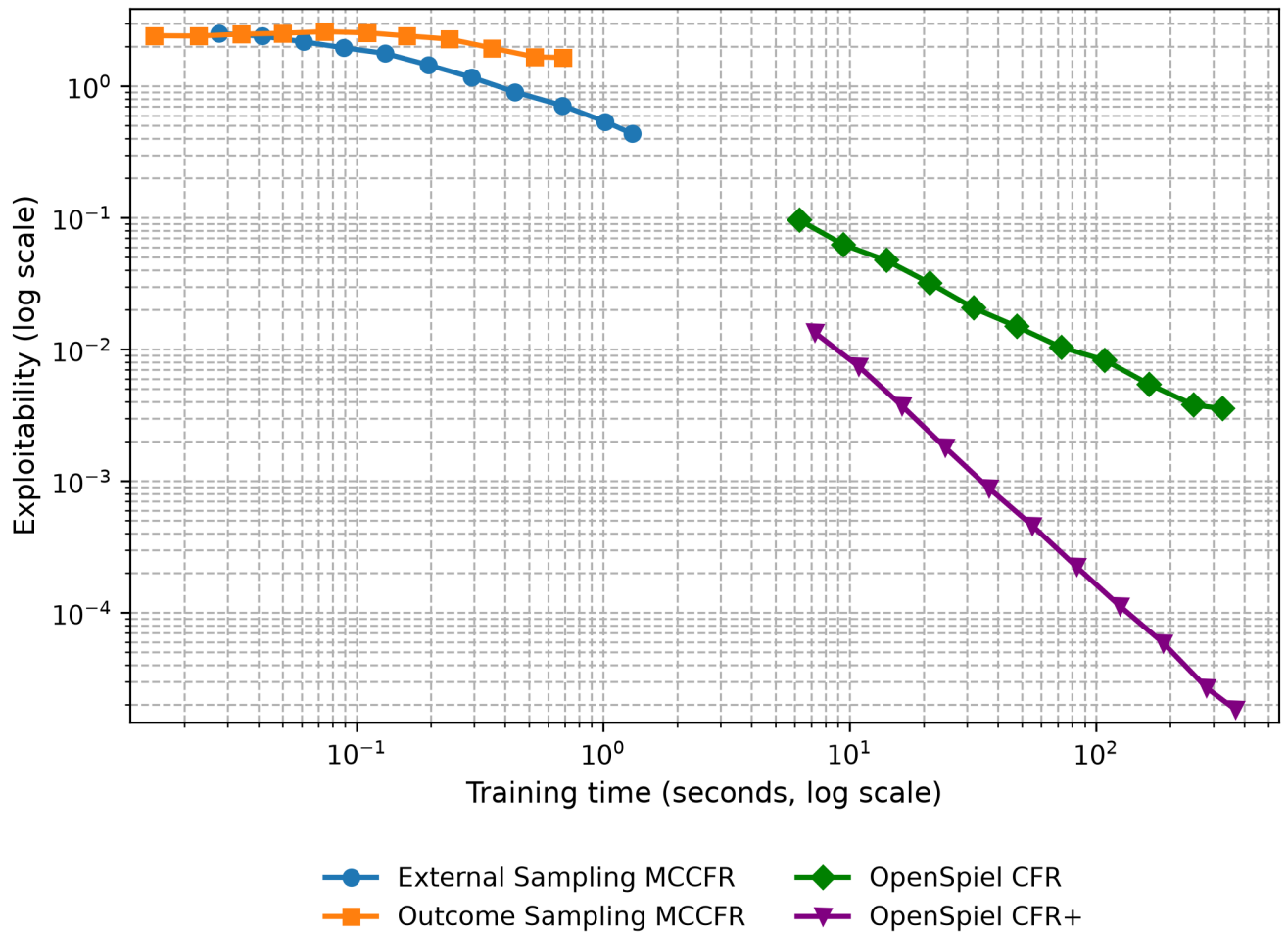


Figure 2: Leduc Poker: exploitability vs training time for the four OpenSpiel solvers

3.6 MCCFR — Sampling the Game Tree

MCCFR replaces full tree traversal with partial sampling. Two variants sit on the same variance-speed curve at different points:

Table 3: External and outcome sampling: what each variant samples at every node type, and the resulting cost per iteration.

Variant	Chance nodes	Traverser's nodes	Opponent's nodes	Cost per iteration
External Sampling	Sample one deal	Explore all actions	Sample one action	~42 nodes (~945× faster than full)
Outcome Sampling	Sample one deal	Sample one action (ε-on-policy)	Sample one action	~5.5 nodes (~2,228× faster)

External Sampling explores every action at the updating player’s nodes but samples at chance and opponent nodes. Regrets for the traversing player are updated based on the sampled subtree. Since only one deal and one opponent path are visited, each iteration touches a tiny slice of the tree.

Outcome Sampling pushes sampling further: a single root-to-terminal trajectory is drawn. Because actions at the updating player’s nodes are also sampled, the regret update must be corrected by **importance sampling** — the ratio of the true reach probability to the sampling probability. An ε -on-policy mixture (with probability ε , choose uniformly; otherwise follow the current strategy) ensures all actions are explored even when the current strategy assigns zero probability.

The critical theoretical property for both: sampled counterfactual values are **unbiased estimators** of the true values (Lanctot et al., Lemma 1). Together with their regret bounds (Theorems 4–5), this guarantees convergence to an approximate Nash equilibrium despite the sampling noise. The cost, as always with Monte Carlo, is variance. Each sampled update deviates from the true counterfactual value because it reflects only one possible deal, not the expectation over all deals.

On Kuhn Poker’s 12 information sets every algorithm runs in seconds, and the ordering is already Leduc’s: after 5,000 iterations CFR+ is at exploitability $\approx 3 \times 10^{-5}$ and vanilla CFR at $\approx 2 \times 10^{-4}$, while external and outcome sampling are still at ≈ 0.013 and ≈ 0.04 (the chance-sampled Chapter 2 solver, which draws one deal per iteration, sits between vanilla CFR and external sampling at ≈ 0.006):

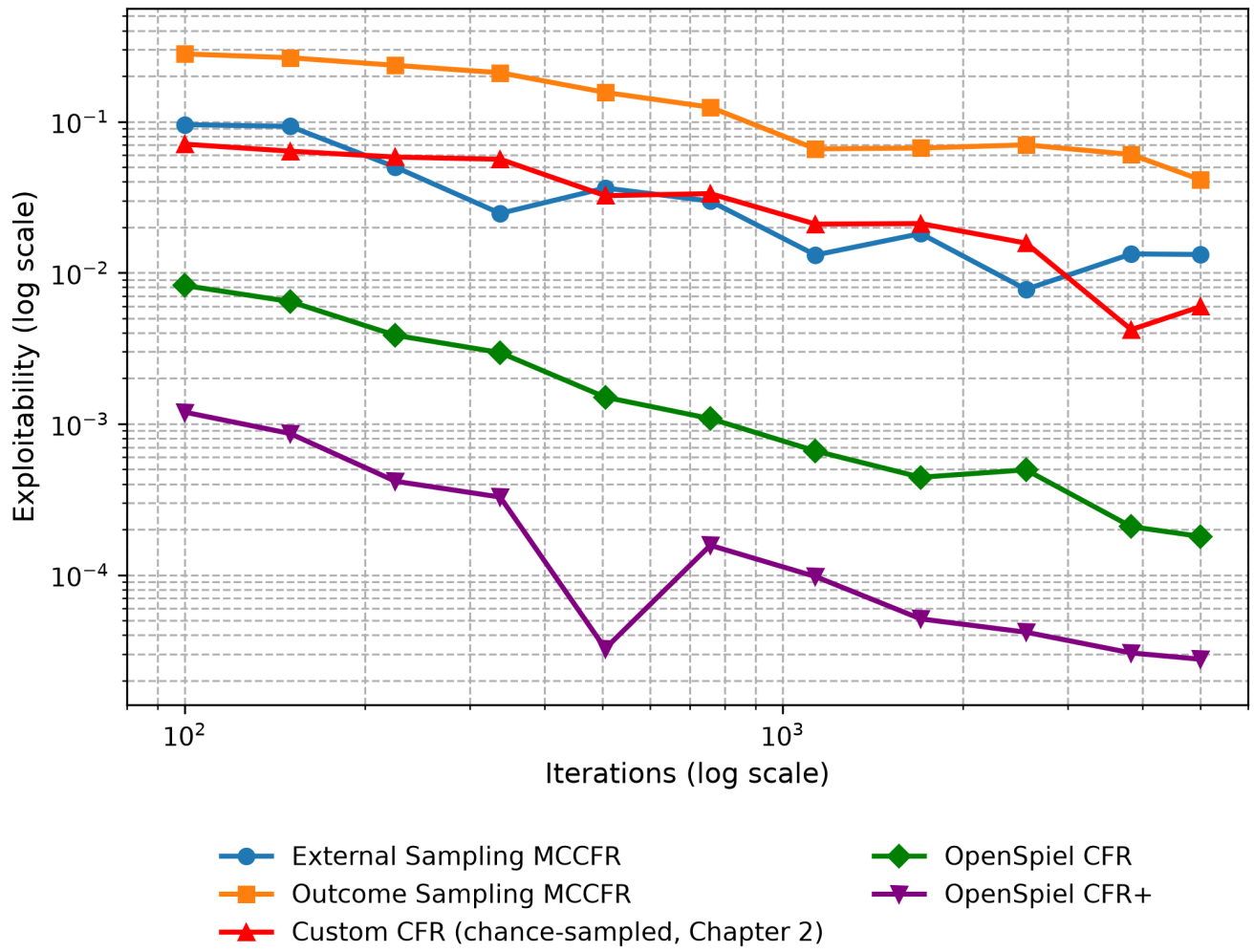


Figure 3: Kuhn Poker: exploitability vs iterations

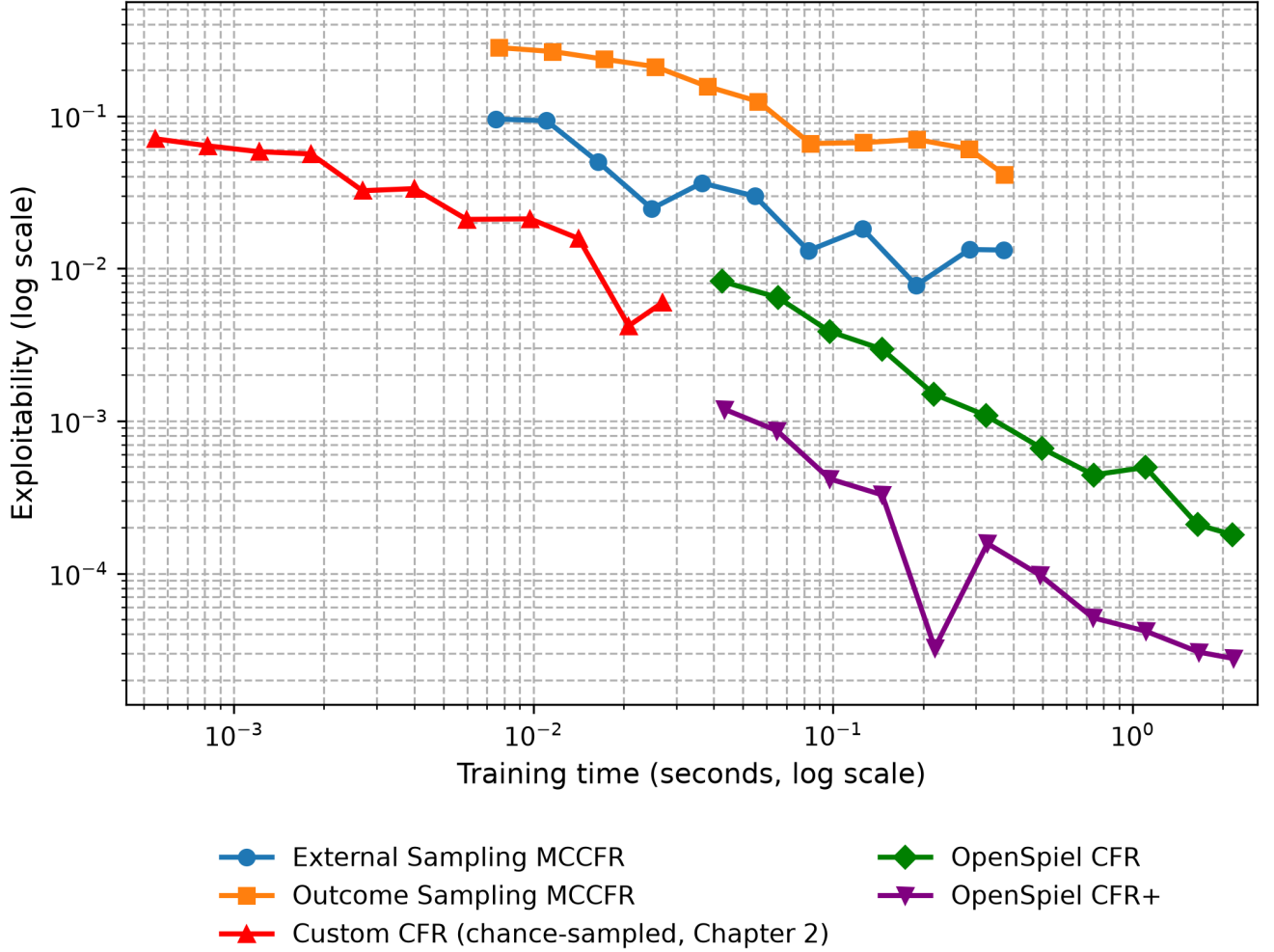


Figure 4: Kuhn Poker: exploitability vs training time

To see the tradeoff emerge, the game tree has to be large enough that per-iteration cost matters. That is Leduc’s role.²

3.7 The Variance-Speed Tradeoff

All four algorithms have a guaranteed $O(1/\sqrt{T})$ upper bound on exploitability — CFR,¹⁰ MCCFR with probability at least $1 - p$,² and CFR+.⁸⁹ Their measured rates differ: both MCCFR variants follow a slope of about -0.4 to -0.5 , vanilla CFR about -0.8 and CFR+ about -1.8 . The model $\epsilon(T) = C/\sqrt{T}$ below is therefore an approximation (for vanilla CFR, $\epsilon\sqrt{T}$ falls from 0.94 to 0.27) and does not apply to CFR+. Converting to wall-clock via $C_w = C/\sqrt{\text{speed}}$:

¹⁰Zinkevich, M., Johanson, M., Bowling, M. & Piccione, C. (2007). “Regret Minimization in Games with Incomplete Information.” *Advances in Neural Information Processing Systems* 20.

Table 4: Measured convergence constants for the four solvers, per iteration and in wall-clock terms.

Algorithm	C_{iter}	Speed (iter/s)	C_w (wall-clock)	C_w relative to vanilla
Vanilla	0.34	20.6	0.075	1.0×
CFR				
CFR+	–	20.6	–	not $C/\sqrt{T}$ (see text)
MCCFR	107	19,470	0.767	10.2× ($\approx 105\times$ more time)
External				
MCCFR	245	45,901	1.143	15.3× ($\approx 233\times$ more time)
Outcome				

The wall-clock constant is what determines real-world performance. Despite being $945\times$ faster per iteration, External Sampling’s variance constant (107 vs 0.34) is $315\times$ larger. The speed advantage **does not overcome the variance penalty**:

$$\frac{99,000 \times \text{more iterations needed}}{945 \times \text{faster per iteration}} \approx 105 \times \text{more wall-clock time}$$

Why does the variance exist? Vanilla CFR computes **exact** counterfactual values by enumerating all 120 deals. Its regret update has zero variance. MCCFR samples one deal per iteration. Even if it explored the full subtree for that deal, it would still have variance across deals — a consequence of sampling the chance event (the deal) itself, not of how the tree is explored within a deal:

$$\text{Var}[\hat{v}_I] = \mathbb{E}_{\text{deal}} [(\hat{v}_I - v_I)^2]$$

A formal 180-second timed benchmark of all four variants on Leduc makes the wall-clock comparison concrete. Per-iteration view (sampling variants complete millions of updates, full-traversal variants a few thousand):

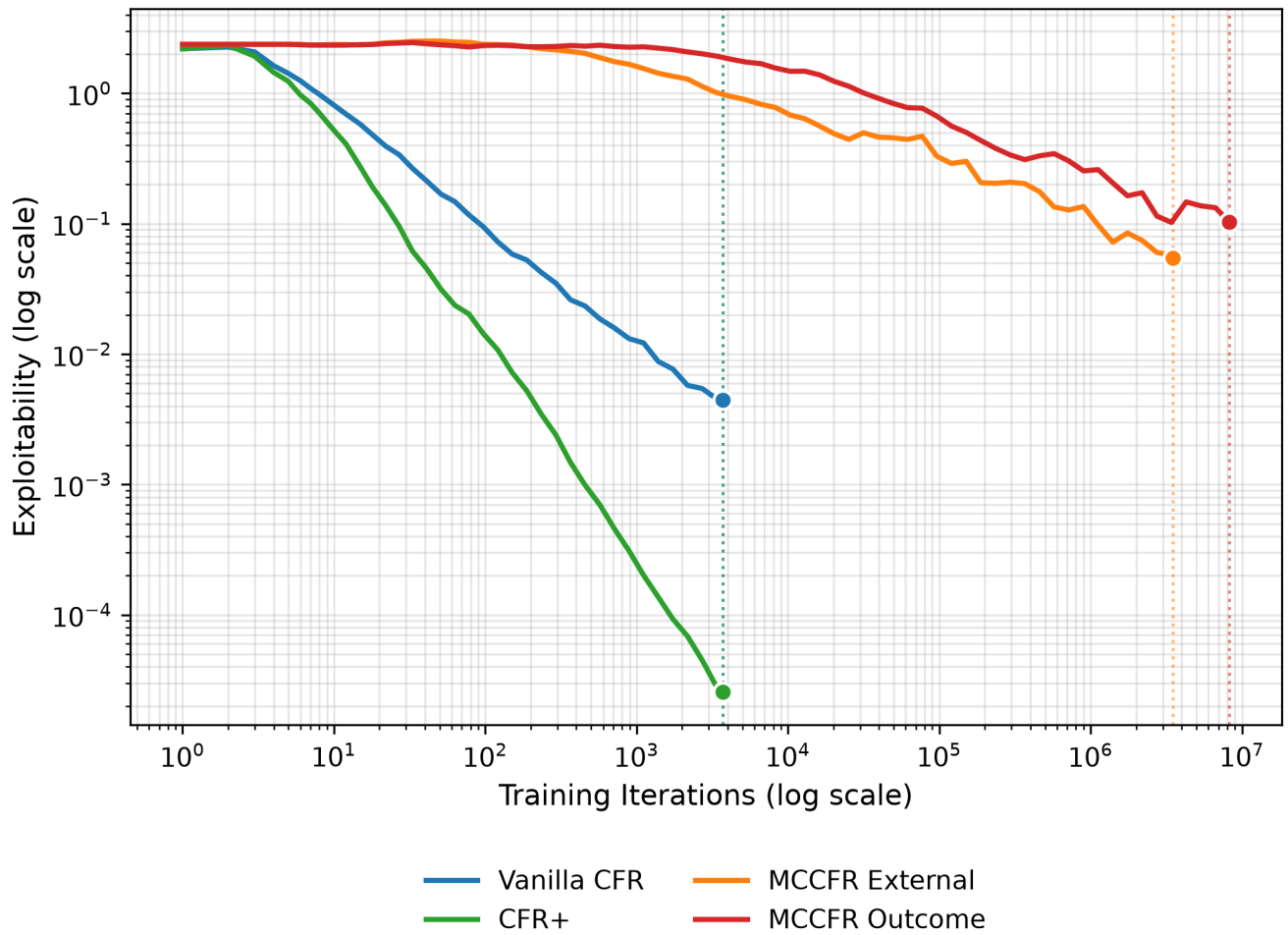


Figure 5: Leduc Poker, own implementations with a 180 s budget per algorithm: exploitability vs iterations

Wall-clock view — the fair comparison (given equal compute, which algorithm produces the best strategy?):

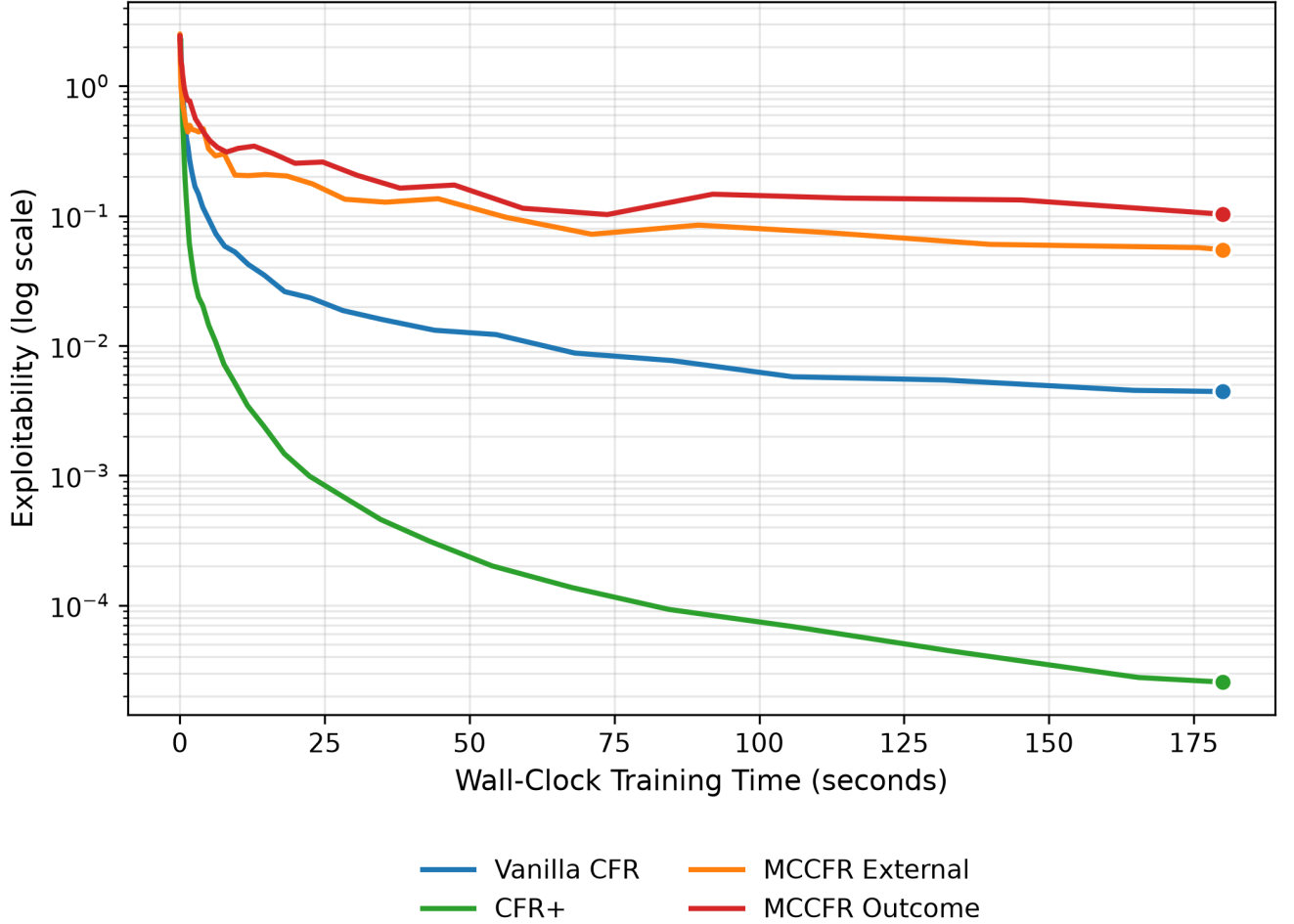


Figure 6: Leduc Poker, own implementations with a 180 s budget per algorithm: exploitability vs wall-clock time

CFR+ reaches exploitability 2.6×10^{-5} — nearly exact Nash — in 3 minutes. Vanilla CFR follows at 4.4×10^{-3} . Both MCCFR variants trail vanilla CFR by 12–23 $\times$ and CFR+ by three to four orders of magnitude on this game size.

3.8 When MCCFR Wins — The Crossover Point

The pattern above raises an obvious question: if MCCFR is always worse on Leduc, why was it invented? The answer is that vanilla CFR’s per-iteration cost grows linearly in the tree size $|N|$, while MCCFR’s grows far more slowly (roughly as $\sqrt{|N|}$ for external sampling²). At sufficient scale the linear cost wins. The critical tree size is:

$$|N|_{\text{crossover}} = |N|_{\text{Leduc}} \cdot \left(\frac{C_{mc}}{C_v} \right)^2 \cdot \frac{\text{speed}_v}{\text{speed}_{mc}}$$

Plugging in measured values:

Table 5: Game size at which MCCFR overtakes vanilla CFR, per sampling variant.

Variant	$ N _{\text{crossover}}$
External Sampling	~1.1 million nodes
Outcome Sampling	~2.4 million nodes

Compared against actual games:

Table 6: The crossover threshold applied to three real games.

Game	$ N $	External wins?	Outcome wins?
Leduc Poker	10,200	No (105× too small)	No (233× too small)
Heads-up limit Texas Hold'em	$\sim 3 \times 10^{17}$	Yes by the model ($\sim 10^{11}$ × above), yet solved by full-traversal CFR+	Yes by the model
No-Limit Texas Hold'em	up to $\sim 6 \times 10^{164}$	Yes	Yes

Leduc sits 105× below the External Sampling crossover and 233× below the Outcome Sampling crossover. Full traversal dominates because the entire game tree fits in memory and can be enumerated in milliseconds. The estimate is an order-of-magnitude figure for this implementation: the constants are measured on Leduc only, vanilla CFR is modelled as $C/\sqrt{T}$ although it converges faster, and if MCCFR’s cost per iteration grows as $\sqrt{|N|}$ the same data put the threshold near 10^8 nodes.

At real poker scales the conclusion reverses, though not uniformly. Heads-up limit hold'em (3.16×10^{17} states) was still solved by full-traversal CFR+ — 1,579 iterations of about 61 minutes each on 4,800 cores³ — and its authors report CFR+ needing considerably less computation than sampling CFR. No-limit hold'em (up to 6.31×10^{164} states in the ACPC formats¹¹) is beyond any full traversal; there sampling — MCCFR combined with abstraction and variance-reduction extensions¹² — is the practical route.

¹¹Johanson, M. (2013). “Measuring the Size of Large No-Limit Poker Games.” arXiv:1302.7008.

¹²Schmid, M., Burch, N., Lanctot, M., Moravčík, M., Kadlec, R. & Bowling, M. (2019). “Variance Reduction in Monte Carlo Counterfactual Regret Minimization (VR-MCCFR) for Extensive Form Games Using Baselines.” *Proc. AAAI*, 33(01), 2157–2164.

3.9 Connections to Chapter 2 and Forward Pointers

Same principle, different regime. Chapter 2 established full-traversal CFR on Kuhn and showed that minimizing regret locally at each information set drives the global strategy to Nash equilibrium. Chapter 3 preserves that principle but introduces two independent refinements: CFR+ modifies how regrets accumulate (flooring + linear weighting), and MCCFR modifies how regrets are measured (sampled estimator instead of exact expectation). Both still output the **average** strategy, not the final iteration — the same averaging mechanism that stabilizes vanilla CFR stabilizes every variant.

Convergence rates. Chapter 2 measured a log-log slope near -0.5 for Kuhn’s exploitability, confirming $O(1/\sqrt{T})$. Chapter 3 shows that $O(1/\sqrt{T})$ is the family’s upper bound, not its observed rate: on Leduc both MCCFR variants follow a slope of about -0.4 to -0.5 , vanilla CFR converges faster (about -0.8) and CFR+ much faster (about -1.8), without a formal proof. MCCFR stays at $O(1/\sqrt{T})$ with a dramatically larger constant.

Forward:

- **Chapter 4 (Game Abstraction & Scaling)** — attacks scalability from the opposite direction: instead of sampling the tree, reduce the tree itself through state and action abstraction. Combines with MCCFR to make real poker tractable.
- **Chapter 5 (Neural Networks for Imperfect-Information Games)** — replaces tabular strategy storage with neural function approximators, using MCCFR’s sampling framework as the data-generation engine. The variance properties established here explain why Deep CFR samples externally and why its outcome-sampling successor DREAM adds a learned baseline to control variance.
- **Chapter 6 (End-to-End Game AI Architectures)** — builds on both CFR+ and MCCFR for complete agents. The Leduc engine built for this chapter remains the benchmark environment in later chapters.¹³

¹³Brown, N. & Sandholm, T. (2017). “Safe and Nested Subgame Solving for Imperfect-Information Games.” *NeurIPS* — bridges tabular CFR with subgame decomposition for real poker.